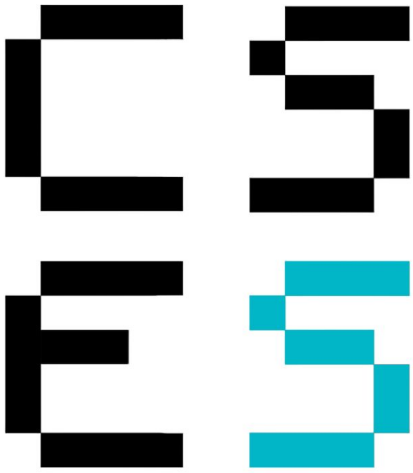
Finnovate

Implementation Guide & Tracklist



Organized by

Computer Science and Engineering Society

INDEX

OVERVIEW

Participants must select one of the following four tracks. Each track includes the formal problem statement, technical requirements, and a "Getting Started" implementation guide designed to help teams with zero prior experience build a functional prototype.

This document is structured to provide a standalone guide for each track. Please navigate to your chosen problem statement for full technical details. For any given problem statement, the technical deliverables represent features you will be objectively graded on and will have the **highest weightage**, however, feel free to implement your own ideas on top of them for extra brownie points.

Proceed to the next page for the Problem Statements.

Problem Statements

TRACK 1: REAL-TIME INSTITUTIONAL TRADE DETECTION

"The Whale Watcher"

1. The Problem Statement

In the volatile crypto market, "Whales" (large-scale holders) influence price movements by moving massive amounts of capital to exchanges to sell. Retail investors are often at a disadvantage because they lack the high-level data tools to see these moves in real-time. By the time a price drop is visible on a standard chart, the move has already happened.

2. The Objective

Build a real-time dashboard that bridges this information gap. The system must maintain a constant connection to blockchain exchange data, filter out "noise" (small trades), and highlight "signals" (massive trades).

3. Technical Deliverables

- **Real-Time Data Pipeline:** A functional WebSocket integration using the Binance stream to pull live BTC/USDT trade data.
- **Automated Alert System:** A logic engine that identifies transactions exceeding $500,000 USD and triggers an immediate visual notification.
- **Analytical Visualizer:** A dynamic line chart plotting trading volume against price over a rolling 60-minute window.
- **Asset Info Module:** Integration with the CoinGecko API to display metadata (logos, symbols).

4. Implementation Guide (How to Build It)

Recommended Stack: Node.js (Backend), Socket.io (Real-time comms), Chart.js (Frontend).

Phase 1: The Data Hose (Backend)

1. **Setup:** Initialize a Node.js project (`npm init -y`) and install `ws` (for Binance) and `socket.io` (for your frontend).
2. **Connect:** Use `ws` to connect to the public Binance stream: `wss://stream.binance.com:9443/ws/btcusdt@trade`
3. **The Filter:** The stream will send JSON objects for every trade. Write a logic check:

- Calculate value: $Price \times Quantity$
- `if (value > 500000)` → `This is a ‘‘Whale.’’`

4. **Broadcast:** When a “Whale” is found, emit an event using Socket.io to the frontend.

Phase 2: The Dashboard (Frontend)

1. **Visuals:** Create an `index.html`. Import Chart.js via CDN.
2. **The Graph:** Initialize a line chart. When the backend sends new price data via Socket.io, push the data to the chart array and call `chart.update()`.
3. **The Alert:** When the backend sends a “Whale” event, trigger a CSS animation (e.g., a flashing red modal) showing the trade size.

Phase 3: Polish

- On page load, use the browser’s `fetch()` to call the CoinGecko API and place the Bitcoin logo next to your header.

TRACK 2: BIOMETRIC-FIRST BANKING

"The Passwordless Portal"

1. The Problem Statement

Text-based passwords are prone to phishing, brute-force attacks, and credential stuffing. As the industry moves toward FIDO2 standards, there is a need for "Passkey" implementation which relies on hardware-level security rather than human memory.

2. The Objective

Eliminate the password field entirely. Create a banking login portal where the "key" is stored in the user's device hardware (TouchID/FaceID). Even if the database is stolen, hackers cannot log in without the physical device.

3. Technical Deliverables

- **Biometric Registration Flow:** A UI allowing users to register their hardware authenticator using the WebAuthn API.
- **Cryptographic Auth Server:** An Express.js backend that generates unique "challenges" and verifies the digital signatures returned by the device.
- **Secure Session Architecture:** A system that issues secure, HTTP-only cookies to manage sessions, preventing XSS attacks.

4. Implementation Guide (How to Build It)

Recommended Stack: Node.js, Express, `@simplewebauthn` library.

Phase 1: The Registration Logic

1. **Library:** Install `@simplewebauthn/server` (backend) and `@simplewebauthn/browser` (frontend).
2. **Challenge:** When a user clicks "Register Device," your server generates a random "challenge" string.
3. **Sign:** The browser prompts the user for a fingerprint/FaceID. The device signs the challenge and creates a public/private key pair.
4. **Store:** The device sends the Public Key to your server. Store this in your database (or a JSON file) associated with the username.

Phase 2: The Login Logic

1. **Request:** User types username. Server looks up the Public Key and sends a new

challenge.

2. **Verify:** The user taps their fingerprint. The device signs the challenge.
3. **Validate:** The server uses the stored Public Key to verify the signature mathematically. If it matches, the user is real.

Phase 3: Security

- If validation passes, use Express to set a cookie:
  `res.cookie('session', token, { httpOnly: true })`
- This ensures JavaScript cannot read the cookie, stopping XSS hackers.

TRACK 3: FRAUD DETECTION API

"The Merchant Shield"

1. The Problem Statement

Small e-commerce businesses are frequently targeted by "Chargeback Fraud." Unlike large corporations, they lack the budget for expensive fraud-prevention suites. They need a lightweight solution to flag high-risk transactions before processing.

2. The Objective

Develop a machine learning microservice that acts as a gatekeeper. By analyzing historical patterns, the model will provide an instant "Risk Score" for incoming transactions.

3. Technical Deliverables

- **Trained ML Model:** A Logistic Regression or Random Forest model trained on the Kaggle Credit Card Fraud dataset.
- **Predictive REST API:** A Python endpoint (`POST /analyze-risk`) that returns a JSON risk assessment.
- **Admin Audit Log:** A dashboard to review transactions flagged as suspicious by the AI.

4. Implementation Guide (How to Build It)

Recommended Stack: Python, Pandas, Scikit-Learn, Flask.

Phase 1: The Brain (Training)

1. **Data:** Download the Credit Card Fraud Detection CSV from Kaggle.
2. **Code:** Use pandas to load the data. Use scikit-learn to split data into training and testing sets.
3. **Train:** Initialize a `LogisticRegression` model. Call `model.fit(X_train, y_train)`. This "teaches" the model.
4. **Save:** Use `joblib` or `pickle` to save the trained model as a file (e.g., `model.pkl`).

Phase 2: The API (Flask)

1. **Setup:** Create a `app.py` file using Flask.
2. **Load:** On startup, load your `model.pkl`.
3. **Route:** Create a generic route. When it receives JSON data (Transaction Amount, Time, V1-V28 features), pass it to `model.predict()`.

4. **Response:** Return the result as JSON: {`"fraud_probability": 0.85`}.

Phase 3: The Interface

- Build a simple HTML form where a user acts as the "Merchant" entering transaction details.
- Display a red warning if the API returns a probability > 0.50.

TRACK 4: SECURE DOCUMENT VAULT

"The Zero-Knowledge KYC"

1. The Problem Statement

Fintechs must collect sensitive KYC documents (e.g., IDs, Tax forms). Storing these in standard cloud buckets is risky; if the database is breached, files are exposed.

2. The Objective

Implement a "Zero-Knowledge" architecture. Encryption must happen on the user's device (client-side). The cloud server should store only encrypted "garbage text" and never possess the decryption key.

3. Technical Deliverables

- **Client-Side Encryption:** A browser module using the Web Crypto API to encrypt files with AES-GCM.
- **Key Derivation:** A system that turns a user's PIN into a key using PBKDF2, ensuring the key never touches the internet.
- **Encrypted Cloud Storage:** Integration with Firebase or AWS S3 to store the blobs.
- **Breach Check:** Integration with the "Have I Been Pwned" API to warn users if their PIN is compromised.

4. Implementation Guide (How to Build It)

Recommended Stack: Vanilla JavaScript (Frontend), Firebase Storage (Cloud).

Phase 1: The Key Generator

1. **Input:** Ask the user for a "Secret PIN" and select a file.
2. **Crypto:** Use the browser's native `window.crypto.subtle`.
3. **Derive:** Use the PBKDF2 algorithm to mix the PIN with a "Salt" (random data) to generate a cryptographic Key. **Crucial:** This key stays in the browser variable; never send it to the server.

Phase 2: The Encryption

1. **Encrypt:** Read the file as an `ArrayBuffer`. Use `window.crypto.subtle.encrypt` with the AES-GCM algorithm and the Key from Phase 1.
2. **Output:** You now have an encrypted "Blob" that looks like random noise.

Phase 3: The Upload

1. **Storage:** Use the Firebase SDK. Upload the Encrypted Blob (not the original file) to the cloud bucket.
2. **Verify:** Check your Firebase console. If you try to view the image, it should be unreadable.

Phase 4: Safety Check

- Before generating the key, hash the PIN and send the first 5 characters to:
  `https://api.pwnedpasswords.com/range/{hash}`
  to check if it's safe.

TRACK 5: PAYMENT CONFUSION DETECTOR

"The Clarity Guardian"

1. The Problem Statement

Payment screens are critical moments where users make financial commitments. Complex checkout flows, hidden fees, or confusing terms can cause users to hesitate by reading and re-reading the same sections repeatedly. This "confusion paralysis" increases cart abandonment. Worse, frustrated users may click through without understanding, leading to disputes and chargebacks later.

2. The Objective

Build a computer vision system that monitors eye movements during checkout. When the AI detects confusion patterns (fixations, erratic saccades, regression reading), it should trigger real-time contextual help, for example by simplifying language, highlighting key information, or offering a human assistant.

3. Technical Deliverables

- **Webcam Eye Tracker:** A JavaScript module using WebGazeJS or TensorFlow.js FaceMesh to extract gaze coordinates in real-time.
- **Confusion Detection Model:** An algorithm that flags confusion based on: dwell time on specific areas, repeated visits to the same text block, rapid eye jumps between distant elements.
- **Adaptive Payment UI:** A mock checkout page that dynamically responds by showing tooltips, expanding summaries, or popping up a "Need Help?" chatbot when confusion is detected.
- **Privacy Dashboard:** Clear consent flow and visualization showing users exactly what eye data is being processed (with a one-click disable).

4. Implementation Guide (How to Build It)

Recommended Stack: JavaScript, WebGazeJS (or TensorFlow.js), HTML/CSS, Chart.js.

Phase 1: The Eye Tracker

1. **Library:** Integrate WebGazeJS (`webgazer.js`). This library uses the webcam to predict where the user is looking on screen.
2. **Calibration:** Prompt the user to look at 9 dots on screen. This trains the model to their unique eye characteristics.
3. **Stream:** Use `webgazer.setGazeListener()` to receive continuous gaze coordi-

nates: {x: 450, y: 320}.

Phase 2: The Confusion Algorithm

1. **Define Zones:** Map your payment UI into regions (e.g., "Price Summary", "Terms Checkbox", "Submit Button").
2. **Track Metrics:** For each zone, log:
   - *Dwell Time:* How long the gaze stays in one zone.
   - *Revisit Count:* How many times the user returns to the same zone.
   - *Saccade Distance:* The pixel distance between consecutive gaze points (high = erratic scanning).
3. **Threshold:** If a zone has > 5 seconds total dwell time AND > 3 revisits, flag it as a "Confusion Hotspot."

Phase 3: The Smart Response

1. **Visual Cues:** When confusion is detected, gently pulse a yellow border around the confusing element and display a tooltip: *"This is your total including tax. Click for breakdown."*
2. **Progressive Disclosure:** Auto-expand collapsed sections (e.g., fee breakdowns) that the user is struggling with.
3. **Human Escalation:** After 15 seconds of sustained confusion, show: *"Would you like to chat with our team?"*

Phase 4: Privacy & Transparency

- **Consent:** Show a clear modal: *"We use your camera to detect where you're looking and provide help. No images are stored. You can disable this anytime."*
- **Indicator:** Display a small eye icon in the corner that turns green when tracking is active. Clicking it pauses the system.
- **Data:** Process everything locally in the browser on the client and never send gaze data to a server.

Phase 5: Validation Dashboard for testing

- Create a heatmap visualization showing where users looked most during checkout.
- Display metrics: Average time to complete payment, confusion events triggered, conversion rate before/after help was shown.

TRACK 6: API ABUSE DETECTION PLATFORM

"The Sentinel Protocol"

1. The Problem Statement

FinTech platforms depend heavily on APIs for core operations such as payments, account access, and transaction processing. These APIs are frequent targets of abuse, including request flooding, credential misuse, and unauthorized access attempts. Traditional monitoring focuses on uptime and latency, offering limited visibility into security threats at the API level.

2. The Objective

Build a centralized monitoring platform that detects, visualizes, and mitigates abnormal or malicious API usage patterns in FinTech systems.

3. Technical Deliverables

- **Simulated FinTech APIs:** Mock endpoints for balance inquiry, transaction processing, and transaction history.
- **Activity Logging System:** Track request frequency, endpoint usage, and authentication success/failure rates.
- **Anomaly Detection Engine:** Identify excessive request rates, repeated failed authorization attempts, and unusual endpoint access patterns.
- **Monitoring Dashboard:** Web interface displaying API usage trends, flagged suspicious activity, and affected clients or tokens.
- **Mitigation Controls:** Rate limiting and temporary access blocking mechanisms.

4. Implementation Guide (How to Build It)

Recommended Stack: Node.js, Express, React, MongoDB, Redis.

Phase 1: API Simulation & Logging

1. **Mock APIs:** Create three Express endpoints:
    - `GET /api/balance` - Returns account balance
    - `POST /api/transaction` - Processes payment
    - `GET /api/history` - Returns transaction list
2. **Middleware:** Add logging middleware that captures: timestamp, client IP, endpoint, HTTP method, status code, response time, API token/key.

3. **Storage:** Store logs in MongoDB with indexed fields for fast querying.

Phase 2: Detection Engine

1. **Define Thresholds:** Set rules for suspicious behavior:
    - More than 100 requests per minute from a single IP
    - More than 5 failed authentication attempts in 10 minutes
    - Accessing endpoints in unusual order (e.g., /transaction without prior /balance check)
2. **Analysis Script:** Create a background job that runs every 60 seconds:
    - Query recent logs from MongoDB
    - Group by client IP and API token
    - Calculate request rates and failure counts
    - Compare against thresholds
3. **Alert Generation:** When a threshold is breached, create an alert document in the database with: client identifier, violation type, timestamp, severity level.

Phase 3: Dashboard & Mitigation

1. **Frontend:** Build a React dashboard with:
    - Line chart showing requests per minute over time
    - Table of recent alerts with severity indicators
    - List of blocked IPs/tokens with unblock buttons
2. **Visualization:** Use a charting library (Chart.js or Recharts) to display API usage trends.
3. **Rate Limiting:** Implement using Redis:
    - On each request, increment a counter: `INCR rate:limit:{clientIP}`
    - Set expiry: `EXPIRE rate:limit:{clientIP} 60`
    - If count exceeds threshold, return 429 (Too Many Requests)
4. **Blocking:** Maintain a Redis set of blocked clients: `SADD blocked:clients {clientIP}`. Check membership before processing any request.

Phase 4: Testing

- Write a script that simulates normal traffic (varied timing, successful auth).

- Write another script that simulates attacks (rapid requests, repeated failures).
- Verify that the dashboard correctly flags malicious patterns and applies rate limits.